

1. Create Nginx dockerfile

1. Set the base image to Ubuntu
2. Install Nginx
3. Install necessary tools: vim wget curl net-tools
4. Remove the default Nginx configuration file
5. Copy a configuration file from the current directory
6. Expose ports (80)
7. Set the default command to execute when creating a new container

Ans:

```
FROM ubuntu:22.04

RUN apt update

RUN apt install -y nginx \
    vim \
    wget \
    curl \
    net-tools

RUN rm /etc/nginx/sites-enabled/default

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

This is the Nginx Dockerfile , I created based on the above requirements.

2. What is the difference between 'RUN', 'CMD', & 'ENTRYPOINT' in dockerfile?

Ans:

RUN Command: This command is used in Dockerfiles to run files that build image and configure the image . It is a build time command. Each run instruction creates a new layer in the docker image.

CMD Command: This command is used to specify the default command to run when a container is started from the Docker image . If no command is specified during the container startup then this default command is used . It can overridden from the command line when the container is run.

ENTRYPOINT Command : This instruction sets the default executable for the container. Any arguments supplied to the docker run command are appended to the ENTRYPOINT command. It is used when we need a container to always run the same base command and want additional commands at the end. It is useful for turning a container into a standalone executable.

3. How to remove container automatically after it exits?

Ans: To remove container automatically after it exits the command used is :

```
docker run --rm <image>
```

So first I run simply container `docker run hello-world` and then I check the containers using `docker ps -a` it will show the container as stopped but not deleted , then i use `docker run --rm hello-world` and check `docker ps -a` to see if the container is deleted automatically.

```
piyush@CK-piyush: ~  
piyush@CK-piyush:~$ docker run hello-world  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/  
  
piyush@CK-piyush:~$ docker ps -a  
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES  
2d5888d3e3d1   hello-world    "/hello"                8 seconds ago Exited (0) 7 seconds ago           modest_brahmagupta  
ef38e883db56   hello-world    "/hello"                32 hours ago  Exited (0) 32 hours ago           wizardly_wescoff  
piyush@CK-piyush:~$  
  
piyush@CK-piyush:~$ docker run --rm hello-world  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/  
  
piyush@CK-piyush:~$ docker ps -a  
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES  
piyush@CK-piyush:~$
```

4. Create a custom bridge network and run two containers (an app and a database) on it. Configure the app to connect to the database using the container's name as the hostname.

Ans:

To create my custom bridge network I used the command `docker network create my-bridge-network`

```
piyush@CK-piyush: ~  
piyush@CK-piyush:~$ docker network create my-bridge-network  
2bdf1f609df8a5b14b9814fa667c0bda5019d7563179f8f5492aeee2de074205  
piyush@CK-piyush:~$ docker network list  
NETWORK ID          NAME                DRIVER              SCOPE  
09903c581671        bridge             bridge             local  
0b4d2496a309        host              host              local  
2bdf1f609df8        my-bridge-network  bridge            local  
db4130584a3b        none              null              local  
piyush@CK-piyush:~$
```

Now running first database container

```
piyush@CK-piyush: ~  
piyush@CK-piyush:~$ docker run -d \  
> --name db \  
> --network my-bridge-network \  
> -e MYSQL_ROOT_PASSWORD=root \  
> -e MYSQL_DATABASE=testdb \  
> mysql:8  
Unable to find image 'mysql:8' locally  
8: Pulling from library/mysql  
d3202636d61f: Download complete  
edb84af18ccb: Download complete  
le9455481d2b: Download complete  
75882d54c3d8: Download complete  
ed7fb090b90f: Download complete  
5ccfd188136e: Download complete  
7a2e50d8b4d1: Download complete  
4f37333d1be6: Downloading 34.6MB/47.31MB  
952b63b654ec: Download complete  
9c2c353aa8db: Downloading 57.67MB/131.1MB  
92ab5109edcc: Download complete  
a8ffb24fde15: Download complete
```

Now running the app container as ubuntu

```
root@82fd792b0a3f: /
piyush@CK-piyush:~$ docker run -it \
> --name app \
> --network my-bridge-network \
> ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
01d7766a2e4a: Pull complete
fd8cda969ed2: Download complete
Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
root@82fd792b0a3f: /#
```

Now finally checking the connection by connecting to the database using the db hostname.

```
root@82fd792b0a3f: /
Setting up mysql-common (5.8+1.1.0build1) ...
update-alternatives: using /etc/mysql/my.cnf.fallback to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Setting up libbsd0:amd64 (0.12.1-1build1.1) ...
Setting up libedit2:amd64 (3.1-20230828-1build1) ...
Setting up mysql-client-core-8.0 (8.0.45-0ubuntu0.24.04.1) ...
Setting up mysql-client-8.0 (8.0.45-0ubuntu0.24.04.1) ...
Setting up mysql-client (8.0.45-0ubuntu0.24.04.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
root@82fd792b0a3f: /# mysql -h db -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

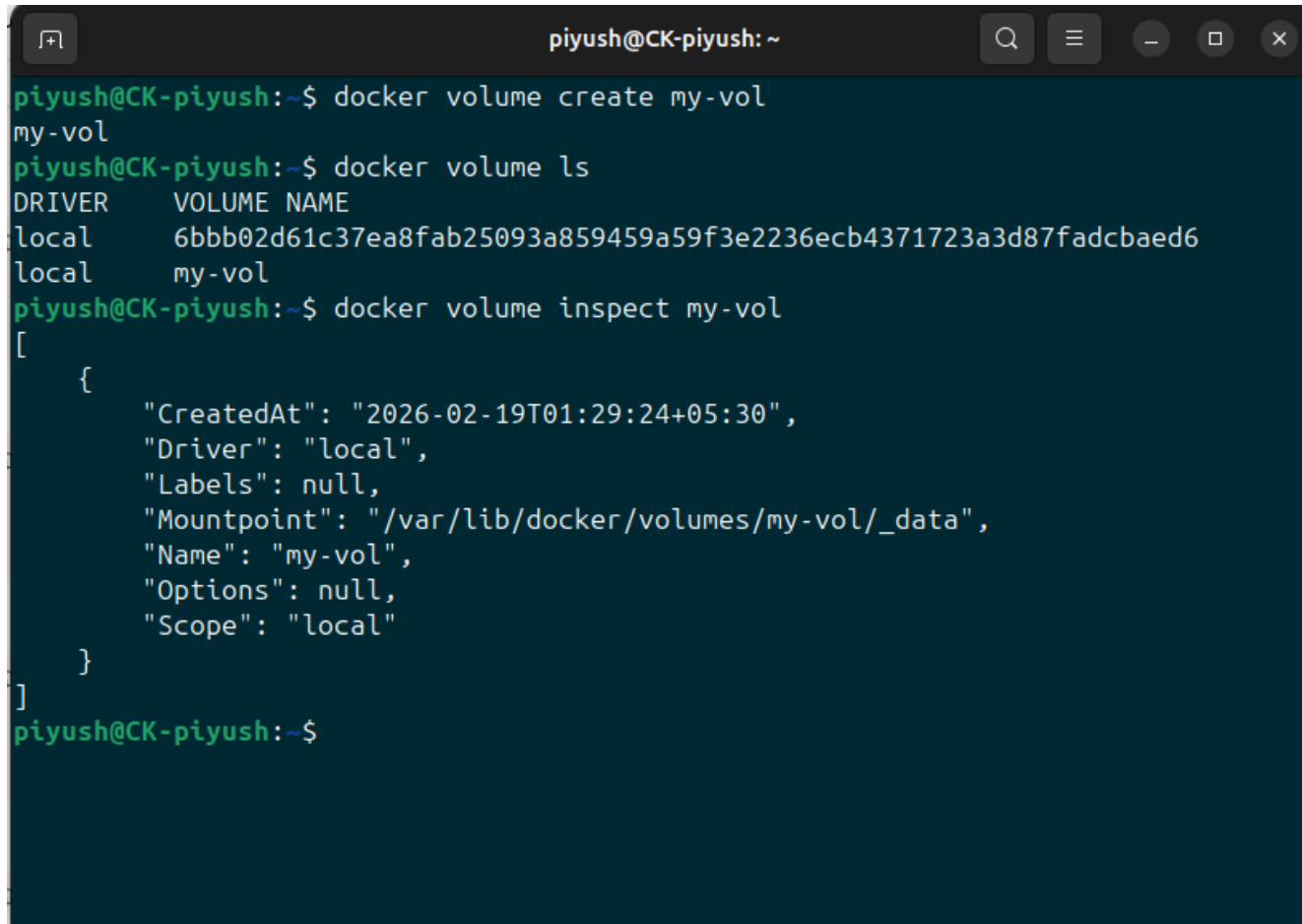
mysql>
```

5. Run a database (e.g., MySQL or PostgreSQL) in a Docker container and use a named volume to ensure the data

persists even after the container is stopped or removed.

Ans: First I create a named volume using the following command

`docker volume create my-vol` and verify it using `docker volume ls`

A terminal window titled 'piyush@CK-piyush: ~' with standard window controls. It shows the execution of three Docker commands: 'docker volume create my-vol', 'docker volume ls', and 'docker volume inspect my-vol'. The output of 'ls' shows a local volume named 'my-vol' with a long ID. The output of 'inspect' shows a JSON object with details about the volume's creation, driver, and mountpoint.

```
piyush@CK-piyush:~$ docker volume create my-vol
my-vol
piyush@CK-piyush:~$ docker volume ls
DRIVER      VOLUME NAME
local       6bbb02d61c37ea8fab25093a859459a59f3e2236ecb4371723a3d87fadcbaed6
local       my-vol
piyush@CK-piyush:~$ docker volume inspect my-vol
[
  {
    "CreatedAt": "2026-02-19T01:29:24+05:30",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/my-vol/_data",
    "Name": "my-vol",
    "Options": null,
    "Scope": "local"
  }
]
```

Now running the mysql database container

`docker run -d \ --name mysql-db \ -e MYSQL_ROOT_PASSWORD=root \ -e
MYSQL_DATABASE=testdb \ -v my-vol:/var/lib/mysql \ -p 3306:3306 \ mysql:8`

```
piyush@CK-piyush: ~  
mysql> USE testdb;  
Database changed  
mysql>  
mysql> CREATE TABLE users (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> name VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql>  
mysql> INSERT INTO users (name) VALUES ('Piyush');  
Query OK, 1 row affected (0.01 sec)  
  
mysql>  
mysql> SELECT * FROM users;  
+----+-----+  
| id | name  |  
+----+-----+  
|  1 | Piyush |  
+----+-----+  
1 row in set (0.00 sec)  
  
mysql>
```

Now checking the database after stopping and removing the container to see that the data

persisted.

```
piyush@CK-piyush:~$ docket stop mysql-db
--name mysql-db-new \
-e MYSQL_ROOT_PASSWORD=root \
-e MYSQL_DATABASE=testdb \
-v my-vol:/var/lib/mysql \
-p 3306:3306 \
mysql:8
20bcd8fac4ecd43deb22da2ea6da08f185c58577b096eef29224d55bfb574466
piyush@CK-piyush:~$ docker exec -it mysql-db-new mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE testdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from users;
+----+-----+
| id | name  |
+----+-----+
|  1 | Piyush |
+----+-----+
1 row in set (0.00 sec)

mysql> 
```

6. You are running a container as a non-root user (UID 1000) and mounting a host directory to /data. The container crashes with "Permission Denied" when writing to that folder. Fix the ownership issue without changing the container to run as root.

Ans: This happens because the bind mount uses the host filesystem to store data so it should have the permissions to write to a file . To fix the issue we can change the ownership of host directory to UID 1000 using `chown` so that the container user has write permission.

```
sudo chown -R 1000:1000 /host/path
```

7. A containerized application works perfectly with docker run, but when you try to scale it to 5 replicas, they all fail because they're trying to bind to port 3000. What's the problem with the container design and how should it be fixed?

Ans: The problem here is that containers were designed to bind a fixed host port i.e 3000 , causing port conflicts when scaling replicas .

By designing the containers to listen only on container ports and using load balancers , reverse proxies ,or orchestrators to distribute traffic without binding the same host port multiple times.

8. Your containerized web application suddenly stops responding after running for a few days. When you check, you find the container is using 8GB of memory even though it should only need 512MB. The container hasn't crashed.

a) What Docker commands would you use to investigate this issue?

b) What Docker features can you configure to prevent this container from consuming excessive memory in production?

Ans:

a) I will use the following commands to see the memory usage and leaks if any
`docker stats` : It will show the live resource usage and will show any excessive memory.
`docker inspect <container-id>` : To see the memory swap allocations
`docker logs <containerid>` : To see the repeated errors logs , retry loops and cache growth logs

b) The following features would come handy to enforce resource limits

- `--memory` flag to set the memory limits for a container
- `--memory-swap` limit to prevent overswap overuse.
- `--oom-kill-disable=false` to kill container safely if memory exceeded.
- `--restart-policy` use the restart policies to auto-recover if killed.

9. A developer's laptop shows 100 stopped containers when running `docker ps -a`. They want to remove all stopped containers but keep running ones. They're unsure whether to use `docker container prune` or manually delete each one. What's the difference and what are the risks?

Ans: The difference between the manual deletion and `docker container prune` is that manual deletion removes selected stopped containers . `docker container prune` removes all stopped containers in one command .

The risks involved are permanent deletion of container data , Loss of debugging artifacts , Cannot restart deleted containers.

10. Your application container needs to perform different tasks based on the environment. Developers want to use `docker run myapp:latest migrate` to run database migrations and `docker run myapp:latest serve` to start the web server. However, the Dockerfile only has `CMD ["node", "server.js"]` which always starts the server. How would you design the container to support multiple runtime commands?

Ans: I would use `ENTRYPOINT+CMD` command to support multiple runtime commands .
By adding `ENTRYPOINT` command in my Dockerfile as

```
ENTRYPOINT ["node"]  
CMD ["server.js"]
```

So when default run happens i.e `docker run myapp` it resolves to `node server.js`
and when `docker run myapp migrate.js` is executed it resolves as `node migrate.js`

Another option I can use is the `entrypoint` script like

```
`#!/bin/sh
```

```
case "$1" in  
serve)  
echo "Starting web server..."  
node server.js  
;;  
migrate)  
echo "Running migrations..."  
node migrate.js  
;;  
*)  
echo "Unknown command"  
exit 1  
;;  
esac  
`
```

and then add in the docker file as

```
`FROM node:18  
  
WORKDIR /app  
COPY . .  
COPY entrypoint.sh /entrypoint.sh  
  
RUN chmod +x /entrypoint.sh
```

```
ENTRYPOINT ["/entrypoint.sh"]
```

```
CMD ["serve"]
```

11. A containerized app listens on port 8080. It works inside the container but is unreachable from the host. Explain why this occurs, differentiate EXPOSE vs -p port publishing, show how to map container port 8080 to (a) host 8080, (b) host 80, and (c) bind only to localhost (not accessible from external network).

Ans :

The app works inside the container but is unreachable from the host because container ports are isolated by default . Host has no route to container port.

EXPOSE : Used in a Dockerfile to indicate which ports the container will listen on at runtime , act as documentation for users of the image , showing which ports are intended to be published , it does not actually publish or open the port to the host , it only marks the port as exposed inside Docker.

-p : Used at container runtime to map a container port to a port on the host . It actually makes the container's port accessible from the host or external network.

(a) `docker run -p 8080:8080 myapp`

(b) `docker run -p 80:8080 myapp`

(c) `docker run -p 127.0.0.1:8080:8080 myapp`

12. Given the following project structure and the Dockerfile:

Rewrite the Dockerfile to produce a minimal production image that excludes dev dependencies, source files, and tests, while serving the built app on port 3000.

Ans:

Open ▾ 

```
FROM node:19 AS builder
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
RUN npm run build
```

```
FROM node:19-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install --only=production
```

```
COPY --from=builder /app/dist ./dist
```

Open ▾ 

```
COPY --from=builder /app/dist ./dist
```

```
COPY --from=builder /app/public ./public
```

```
EXPOSE 3000
```

```
CMD ["node", "dist/server.js"]
```